

CS2610: COMPUTER ORGANIZATION AND ARCHITECTURE

Lab 12 Report

Screenshots Numbering-

1. Original_vectorization_compiler_vectorization
 - Screenshot Numbered 1-9 :- vectorization inspection and compiler vectorisation
 - Screenshot perf :- perf screenshot
 - Screenshot number 10-12 :- Input size sweep
2. Interchanged_vectorization_compiler_vectorization
 - Screenshot Numbered 1-9 :- vectorization inspection and compiler vectorisation
 - Screenshot perf :- perf screenshot
 - Screenshot numbered 10-13 :- Input size sweep

QUESTIONS

Q1. REPORT AND COMPARISON

KERNEL	VERSION	CYCLES	INSTRUCTIONS	IPC	CPI	CACHE REF	CACHE MISSES
Matmul	Original	3821551	54969635	1.5786	0.6335	305584	197025
Matmul	Interchanged	15122594	49687151	3.2856	0.3044	84594	48792
Transpose	Original	10350117	11239224	1.0859	0.9209	341937	95363
Transpose	Interchanged	5698796	3073897	0.5394	1.8539	258241	49084

Q2. ANALYSIS QUESTIONS

(a) *How does Loop interchange affect locality?*

Ans- Loop interchange changes the order in which memory is accessed, which directly impacts spatial and temporal locality. It improves spatial locality by aligning memory access with row-major storage of the arrays.

(b) *Which ordering improves cache behaviour?*

Ans- For matmul, the i-k-j ordering i.e the interchanged ordering is better as it allows reuse of $A[i][k]$ and better access patterns for $B[k][j]$. It is also evident from the number of cache misses dropping from 197025 to 48792. For transpose, nothing really matters as transpose is inherently cache-unfriendly due to column-wise writes.

(c) *How does IPC change after transformation?*

Ans- IPC has increased after loop interchanged as there is better cache locality which results in fewer cache misses and fewer pipeline stalls. This also ultimately results in better instruction execution by the CPU. Hence, IPC has increased. The data also proves so.

(d) *Does instruction count change significantly? Why?*

Ans- Instruction count didn't change significantly in matmul. It decreased slightly probably due to compiler optimizations. Otherwise, it doesn't change that much as Loop interchange only changes execution order, not the computation itself. But, in transpose, it dropped from 11239224 to 3073897, as the optimized pattern will allow the compiler to eliminate the redundant address calculations.

(e) *Which kernel is more sensitive to loop ordering?*

Ans- Matmul kernel is more sensitive to loop ordering as it involves three nested loops and heavy data reuse. Also, its complexity due to three nested loops will be $O(n^3)$. This compounds the penalty of cache misses

(f) *Does loop interchange shift the kernel toward compute-bound or memory-bound behaviour?*

Ans- Yes, the loop interchange shifts the behaviour. It shifts the kernel towards Compute-bound behaviour. It does so by lowering the number of cache misses. This way the processor is only limited by the speed with which it can execute the instructions and not the data.

NANDINI JAIN
CS24B035